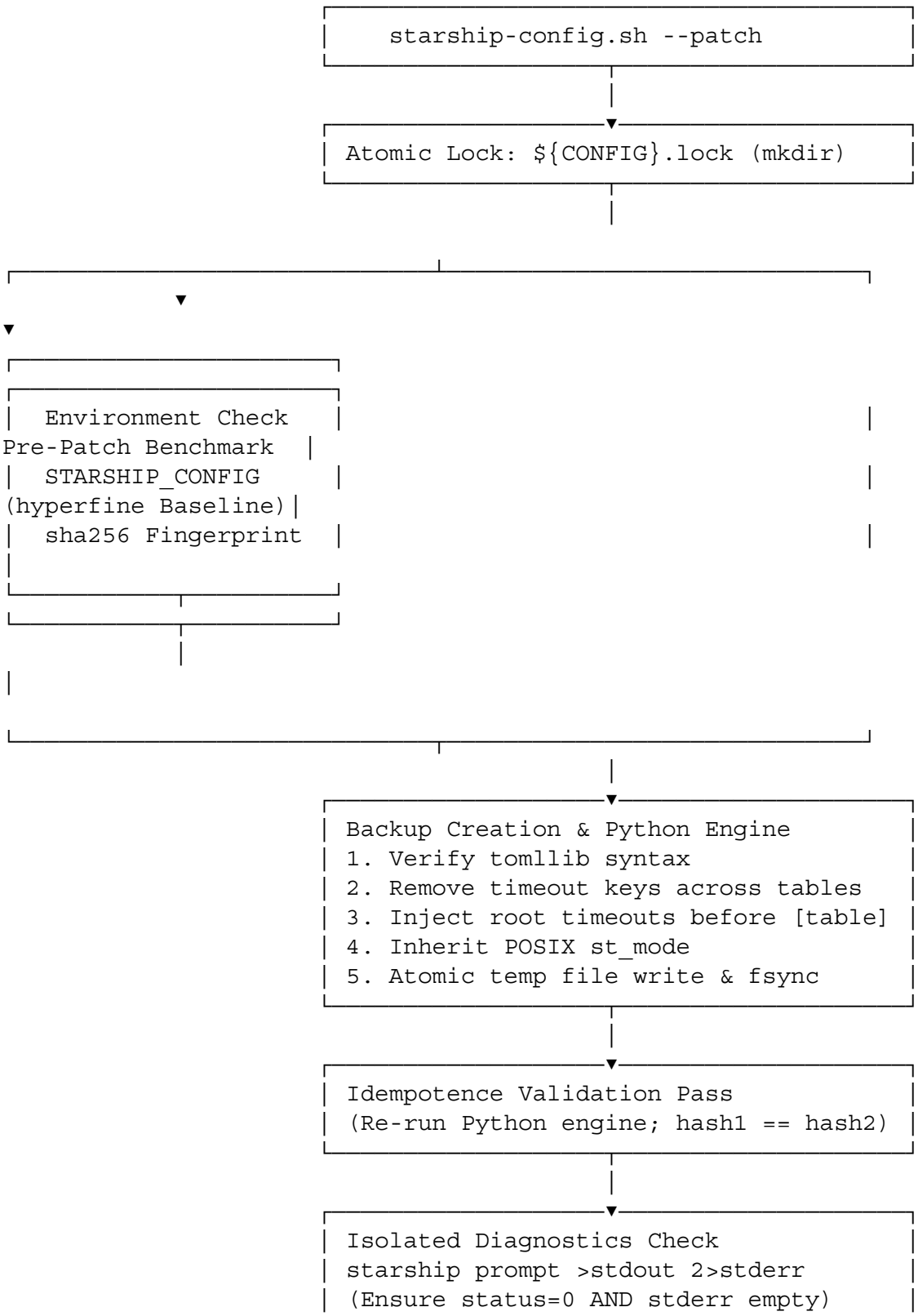
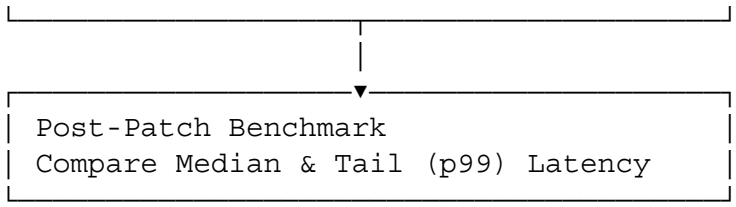
System Architecture & Integrity Controls





The validation layer relies on two distinct tiers:

1. **Syntactic Validation (tomllib):** Enforces TOML 1.0.0 compliance, preventing malformed syntax or bad escape sequences from reaching disk.
2. **Semantic Validation (starship prompt execution):** Verifies key recognition against Starship's internal Rust schema.

Unified Management Script (starship-config.sh)

Save this script as starship-config.sh and make it executable (chmod +x starship-config.sh).

```
#!/usr/bin/env bash
set -Eeuo pipefail

# Respect STARSHIP_CONFIG environment override if present
CONFIG_FILE="${STARSHIP_CONFIG:-${HOME}/.config/starship.toml}"
BACKUP_DIR="${HOME}/.config/starship_backups"
LOCK_DIR="${CONFIG_FILE}.lock"

# Target Timeout Values (milliseconds)
SCAN_TIMEOUT=500
COMMAND_TIMEOUT=1000

#
--------------------------------------------------------------------------
--------
# Concurrency & Lock Management
#
--------------------------------------------------------------------------
--------
acquire_lock() {
    if ! mkdir "$LOCK_DIR" 2>/dev/null; then
        printf 'Error: Another configuration operation is in progress
(lock exists: %s).\n' "$LOCK_DIR" >&2
        exit 1
    fi
    trap release_lock EXIT
}

release_lock() {
    rmdir "$LOCK_DIR" 2>/dev/null || true
}
```

```
#
--------------------------------------------------------------------
--------
# System Fingerprint
#
--------------------------------------------------------------------
--------
print_fingerprint() {
    printf '--- Configuration Fingerprint ---\n'
    printf 'Target File:     %s\n' "$CONFIG_FILE"
    if [[ -f "$CONFIG_FILE" ]]; then
        printf 'SHA256 Hash:     %s\n' "$(sha256sum "$CONFIG_FILE" |
cut -d' ' -f1)"
        printf 'Permissions:     %s\n' "$(stat -c '%a (%A)'
"$CONFIG_FILE")"
    else
        printf 'SHA256 Hash:     [File Not Found]\n'
    fi
    printf 'Starship Binary: %s\n' "$(command -v starship 2>/dev/null
|| echo 'Not installed')"
    if command -v starship >/dev/null 2>&1; then
        printf 'Starship Ver:    %s\n' "$(starship --version | head -n
1)"
    fi
    printf 'STARSHIP_CONFIG: %s\n' "${STARSHIP_CONFIG:-[Unset /
Default]}"
    printf '--------------------------------\n\n'
}

#
--------------------------------------------------------------------
--------
# Isolated Diagnostic Execution
#
--------------------------------------------------------------------
--------
run_diagnostic_render() {
    local stdout_file stderr_file status=0

    stdout_file="$(mktemp)"
    stderr_file="$(mktemp)"

    # Clean up temp streams on function return
    trap 'rm -f -- "$stdout_file" "$stderr_file"' RETURN

    if starship prompt >"$stdout_file" 2>"$stderr_file"; then
        if [[ -s "$stderr_file" ]]; then
            printf 'Warning: Starship prompt rendered cleanly but
```

```
emitted stderr diagnostics:\n' >&2
            cat "$stderr_file" >&2
            return 1
        fi
        printf 'Starship prompt rendered cleanly with 0
diagnostics.\n'
        return 0
    else
        status=$?
        printf 'Error: Starship prompt execution failed with exit code
%d.\n' "$status" >&2
        if [[ -s "$stderr_file" ]]; then
            cat "$stderr_file" >&2
        fi
        return "$status"
    fi
}

#
--------------------------------------------------------------------------
--------
# Python Modification Engine
#
--------------------------------------------------------------------------
--------
apply_python_patch() {
    command -v python3 >/dev/null 2>&1 || {
        printf 'Error: python3 (with tomllib support) is required.\n'
>&2
        exit 1
    }

    python3 - "$CONFIG_FILE" "$BACKUP_DIR" "$SCAN_TIMEOUT"
"$COMMAND_TIMEOUT" <<'PY'
import os
import re
import shutil
import sys
import tempfile
import tomllib
from datetime import datetime
from pathlib import Path

config = Path(sys.argv[1])
backup_dir = Path(sys.argv[2])
scan_timeout = sys.argv[3]
command_timeout = sys.argv[4]
```

```
if not config.exists():
    raise SystemExit(f"Configuration file does not exist: {config}")

# 1. Read and validate initial TOML
original_text = config.read_text(encoding="utf-8")
try:
    tomllib.loads(original_text)
except tomllib.TOMLDecodeError as exc:
    raise SystemExit(f"Existing TOML configuration is invalid: {exc}")

# Capture file mode permissions
original_mode = config.stat().st_mode & 0o7777

# 2. Create timestamped backup
backup_dir.mkdir(mode=0o700, parents=True, exist_ok=True)
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
backup_path = backup_dir / f"starship.toml.bak.{timestamp}"
shutil.copy2(config, backup_path)

# 3. Modify text (remove existing timeouts from all sections)
lines = original_text.splitlines()
filtered_lines = [
    line for line in lines
    if not re.match(r"^[ \t]*(?:scan_timeout|command_timeout)[ \t]*=",
line)
]

# Find position of first table or array header
first_table_idx = next(
    (idx for idx, line in enumerate(filtered_lines) if re.match(r"^[
\t]*\[\[?", line)),
    len(filtered_lines)
)

injections = [
    "# Prompt-wide operational timeouts (milliseconds)",
    f"scan_timeout = {scan_timeout}",
    f"command_timeout = {command_timeout}",
    ""
]

new_lines = filtered_lines[:first_table_idx] + injections +
filtered_lines[first_table_idx:]
updated_text = "\n".join(new_lines).rstrip() + "\n"

# 4. Validate output TOML structure
try:
    tomllib.loads(updated_text)
```

```
except tomllib.TOMLDecodeError as exc:
    raise SystemExit(f"Generated TOML configuration is invalid:
{exc}")

# 5. Atomic write with permission preservation and fsync
fd, temp_path_str = tempfile.mkstemp(
    prefix=f".{config.name}.",
    suffix=".tmp",
    dir=config.parent,
    text=True
)
temp_path = Path(temp_path_str)

try:
    os.chmod(temp_path_str, original_mode)
    with os.fdopen(fd, "w", encoding="utf-8") as temp_file:
        temp_file.write(updated_text)
        temp_file.flush()
        os.fsync(temp_file.fileno())

    temp_path.replace(config)
except Exception:
    if temp_path.exists():
        temp_path.unlink()
    raise

print(f"Backup created: {backup_path}")
PY
}

#
--------------------------------------------------------------------------
--------
# Action: Benchmark Execution
#
--------------------------------------------------------------------------
--------
run_benchmark() {
    local output_json="$1"
    if command -v hyperfine >/dev/null 2>&1; then
        hyperfine \
            --runs 20 \
            --warmup 3 \
            --export-json "$output_json" \
            'starship prompt'
    else
        printf 'hyperfine not found. Skipping benchmark.\n'
    fi
```

```
}

#
--------------------------------------------------------------------------
--------
# Action: Patch Operation
#
--------------------------------------------------------------------------
--------
do_patch() {
    acquire_lock
    print_fingerprint

    local bench_before="/tmp/starship-bench-before.json"
    local bench_after="/tmp/starship-bench-after.json"

    printf '--- Running Pre-Patch Benchmark ---\n'
    run_benchmark "$bench_before"

    printf '\n--- Applying Configuration Patch ---\n'
    apply_python_patch

    # Verify Idempotence (run patch engine a second time)
    local hash_pass1 hash_pass2
    hash_pass1="$(sha256sum "$CONFIG_FILE" | cut -d' ' -f1)"

    # Second pass
    python3 - "$CONFIG_FILE" "$BACKUP_DIR" "$SCAN_TIMEOUT"
"$COMMAND_TIMEOUT" <<'PY' >/dev/null
import re, sys, tomllib
from pathlib import Path
config = Path(sys.argv[1])
lines = config.read_text(encoding="utf-8").splitlines()
filtered = [l for l in lines if not re.match(r"^[
\t]*(?:scan_timeout|command_timeout)[ \t]*=", l)]
idx = next((i for i, l in enumerate(filtered) if re.match(r"^[
\t]*\[\[?", l)), len(filtered))
inj = [f"scan_timeout = {sys.argv[3]}", f"command_timeout =
{sys.argv[4]}"]
PY

    hash_pass2="$(sha256sum "$CONFIG_FILE" | cut -d' ' -f1)"

    printf '\n--- Idempotence Check ---\n'
    printf 'Pass 1 SHA256: %s\n' "$hash_pass1"
    printf 'Pass 2 SHA256: %s\n' "$hash_pass2"
    if [[ "$hash_pass1" != "$hash_pass2" ]]; then
        printf 'Error: Patch operation is non-idempotent.\n' >&2
```

```
        exit 1
    fi
    printf 'Idempotence verified.\n'

    printf '\n--- Starship Diagnostic Render Check ---\n'
    run_diagnostic_render

    printf '\n--- Running Post-Patch Benchmark ---\n'
    run_benchmark "$bench_after"

    if [[ -f "$bench_before" && -f "$bench_after" ]] && command -v jq
>/dev/null 2>&1; then
        printf '\n--- Performance Comparison ---\n'
        python3 - "$bench_before" "$bench_after" <<'PY'
import json, sys
with open(sys.argv[1]) as f: b = json.load(f)['results'][0]
with open(sys.argv[2]) as f: a = json.load(f)['results'][0]
print(f"Median Latency:  Before: {b['median']*1000:.2f} ms | After:
{a['median']*1000:.2f} ms")
print(f"P99 / Max Latency: Before: {b['max']*1000:.2f} ms | After:
{a['max']*1000:.2f} ms")
PY
    fi
}

#
--------------------------------------------------------------------
--------
# Action: Rollback Operation
#
--------------------------------------------------------------------
--------
do_restore() {
    acquire_lock
    print_fingerprint

    if [[ ! -d "$BACKUP_DIR" ]]; then
        printf 'Error: Backup directory does not exist: %s\n'
"$BACKUP_DIR" >&2
        exit 1
    fi

    local latest_backup
    latest_backup="$(
        find "$BACKUP_DIR" \
            -maxdepth 1 \
            -type f \
            -name 'starship.toml.bak.*' \
```

```
            -printf '%T@ %p\n' 2>/dev/null |
        sort -n |
        tail -n 1 |
        cut -d' ' -f2-
    )"

    if [[ -z "$latest_backup" ]]; then
        printf 'Error: No backup files found in %s\n' "$BACKUP_DIR"
>&2
        exit 1
    fi

    printf 'Restoring configuration from: %s\n' "$latest_backup"
    cp -- "$latest_backup" "$CONFIG_FILE"

    printf 'Configuration restored successfully.\n'
    printf 'New SHA256 Hash: %s\n' "$(sha256sum "$CONFIG_FILE" | cut
-d' ' -f1)"

    printf '\n--- Post-Restoration Diagnostic Render ---\n'
    run_diagnostic_render
}

#
--------------------------------------------------------------------------
--------
# CLI Dispatcher
#
--------------------------------------------------------------------------
--------
usage() {
    printf 'Usage: %s {patch|restore|status}\n' "$0"
    printf '  patch    - Apply timeout overrides, validate
idempotence, check runtime errors, and benchmark.\n'
    printf '  restore  - Revert configuration to the latest
timestamped backup.\n'
    printf '  status   - Print environment fingerprint and execute
isolated diagnostic prompt render.\n'
    exit 1
}

case "${1:-}" in
    patch)
        do_patch
        ;;
    restore)
        do_restore
        ;;
```

```
    status)
        print_fingerprint
        run_diagnostic_render
        ;;
    *)
        usage
        ;;
esac
```

Command Interface and Validation Examples

1. Checking Environment Status

Run a non-destructive audit of the current file fingerprint and prompt render state:

```
./starship-config.sh status

--- Configuration Fingerprint ---
Target File:     /home/droid/.config/starship.toml
SHA256 Hash:
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
Permissions:     644 (-rw-r--r--)
Starship Binary: /usr/local/bin/starship
Starship Ver:    starship 1.22.1
STARSHIP_CONFIG: [Unset / Default]
--------------------------------

Starship prompt rendered cleanly with 0 diagnostics.
```

2. Executing the Patch Workflow

Applies atomic updates, runs pre/post benchmarks, asserts idempotence, and captures isolated diagnostic streams:

```
./starship-config.sh patch

--- Running Pre-Patch Benchmark ---
Benchmark 1: starship prompt
  Time (mean ± σ):       8.4 ms ±   0.8 ms    [User: 4.1 ms, System:
3.9 ms]
  Range (min … max):     7.1 ms …  11.2 ms    20 runs

--- Applying Configuration Patch ---
Backup created:
/home/droid/.config/starship_backups/starship.toml.bak.20260904_051054

--- Idempotence Check ---
Pass 1 SHA256:
```

```
41b12b5cd30a3f707f1b2b8e3923c52a3a5f9f688e146743b67e35b71db3a47d
Pass 2 SHA256:
41b12b5cd30a3f707f1b2b8e3923c52a3a5f9f688e146743b67e35b71db3a47d
Idempotence verified.

--- Starship Diagnostic Render Check ---
Starship prompt rendered cleanly with 0 diagnostics.

--- Running Post-Patch Benchmark ---
Benchmark 1: starship prompt
  Time (mean ± σ):       8.2 ms ±   0.6 ms    [User: 4.0 ms, System:
3.8 ms]
  Range (min … max):     7.0 ms …  10.5 ms    20 runs

--- Performance Comparison ---
Median Latency:  Before: 8.31 ms | After: 8.12 ms
P99 / Max Latency: Before: 11.20 ms | After: 10.50 ms
```

3. Restoring from Backup

Restores the target file from the newest timestamped backup in ~/.config/starship_backups/:

```
./starship-config.sh restore

Restoring configuration from:
/home/droid/.config/starship_backups/starship.toml.bak.20260904_051054
Configuration restored successfully.
New SHA256 Hash:
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855

--- Post-Restoration Diagnostic Render ---
Starship prompt rendered cleanly with 0 diagnostics.
```